Day 5

Wednesday, 19 August 2026 9:11 AM

100. Same Tree

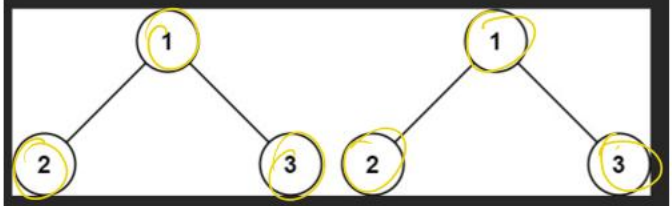
Solved

Easy | Topics | Companies

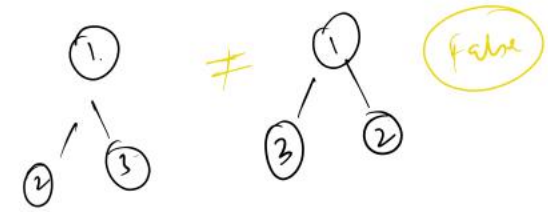
Given the roots of two binary trees `p` and `q`, write a function to check if they are the same or not.

Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:




```
Input: p = [1,2,3], q = [1,2,3]
Output: true
```




```python
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if p is None and q is None :
            return True
        if p is None or q is None or p.val != q.val :
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.
        right)
```

T.C → O(n)

S.C → O(n)

↓

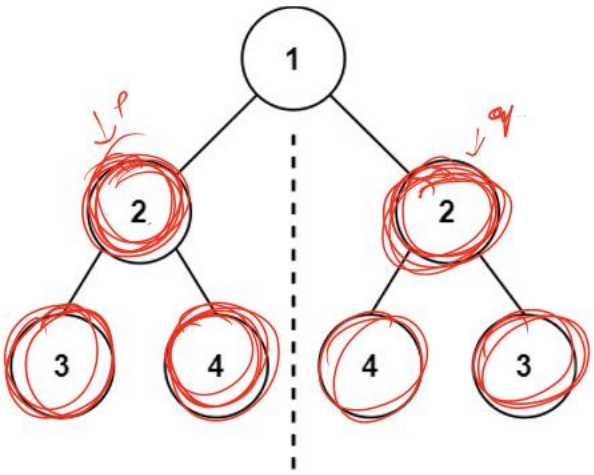
call stack space

101. Symmetric Tree

Solved

Easy | Topics | Companies

Given the `root` of a binary tree, *check whether it is a mirror of itself* (i.e., symmetric around its center).




```python
class Solution:
    def isSymmetric(self, root: Optional[TreeNode]) -> bool:
        def helper(p, q) :
            if p is None and q is None :
                return True
            if p is None or q is None or p.val != q.val :
                return False
            return helper(p.left, q.right) and helper(p.right, q.left)
        return helper(root.left, root.right)
```

T.C → O(n) S.C → O(n) → call stack space

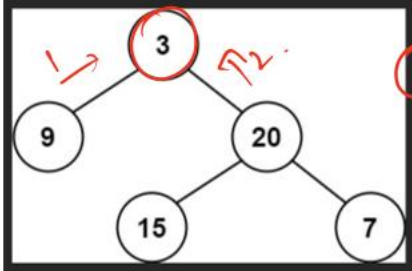
110. Balanced Binary Tree

Solved

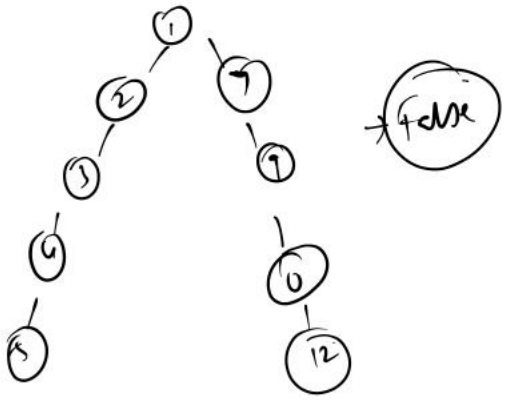
Easy | Topics | Companies

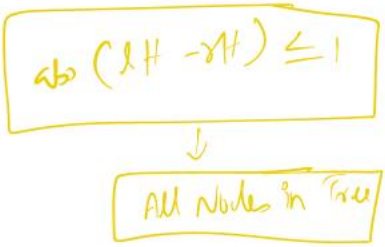
Given a binary tree, determine if it is height-balanced.

Example 1:



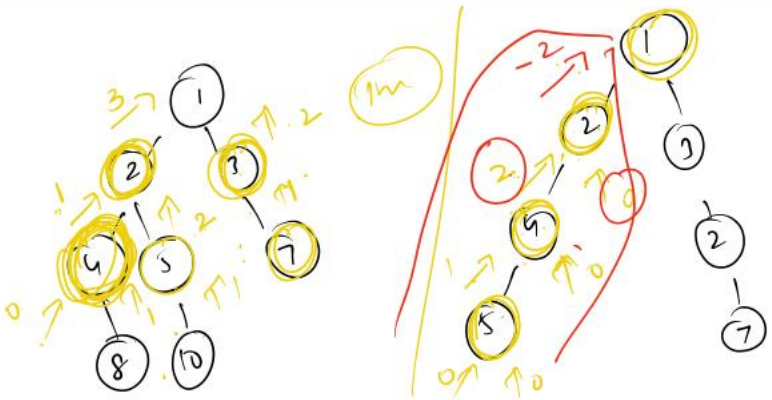

```
Input: root = [3,9,20,null,null,15,7]
Output: true
```






```python
class Solution:
    def height(self, root : Optional[TreeNode]) -> int :
        if root is None :
            return 0
        left = self.height(root.left)
        right = self.height(root.right)
        return 1 + max(left, right)

    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        if root is None:
            return True
        leftHeight = self.height(root.left)
        rightHeight = self.height(root.right)
        if abs(leftHeight - rightHeight) > 1 :
            return False
        return self.isBalanced(root.left) and self.isBalanced(root.right)
```



```python
class Solution:
    def isBalanced(self, root: Optional[TreeNode]) -> bool:
        def helper(root) :
            if root is None :
```

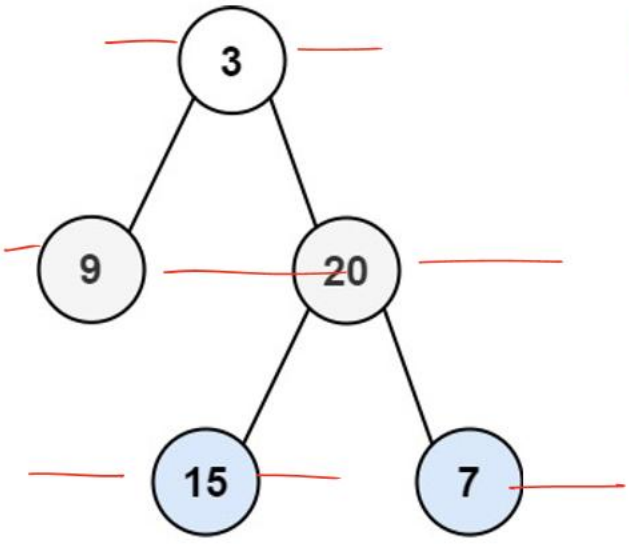
```
        return 0
    leftHeight = helper(root.left)
    if leftHeight == -2 :
        return -2
    rightHeight = helper(root.right)
    if rightHeight == -2 :
        return -2
    if abs(leftHeight - rightHeight) > 1 :
        return -2
    return 1 + max(leftHeight, rightHeight)
return helper(root) != -2
```

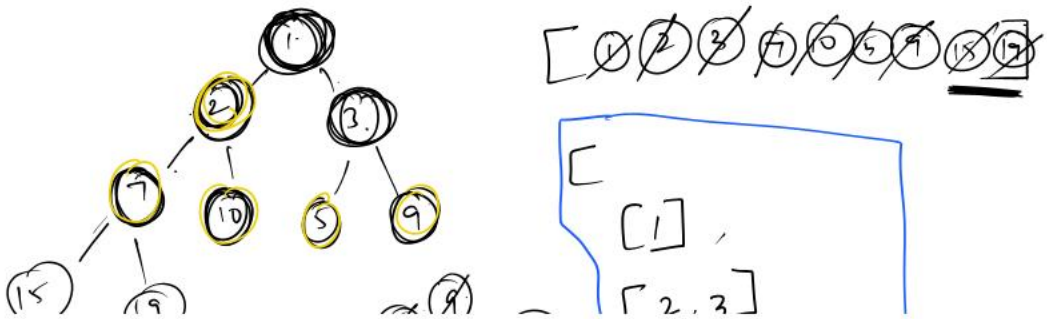
102. Binary Tree Level Order Traversal

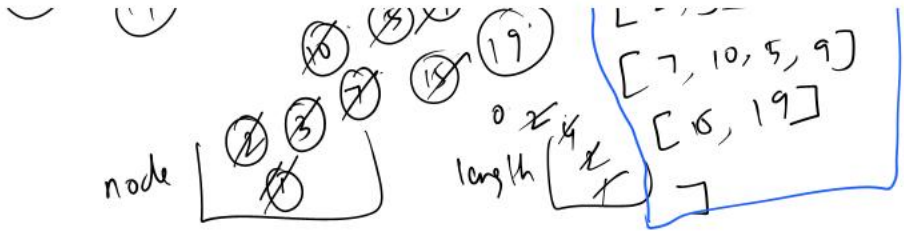
Solved

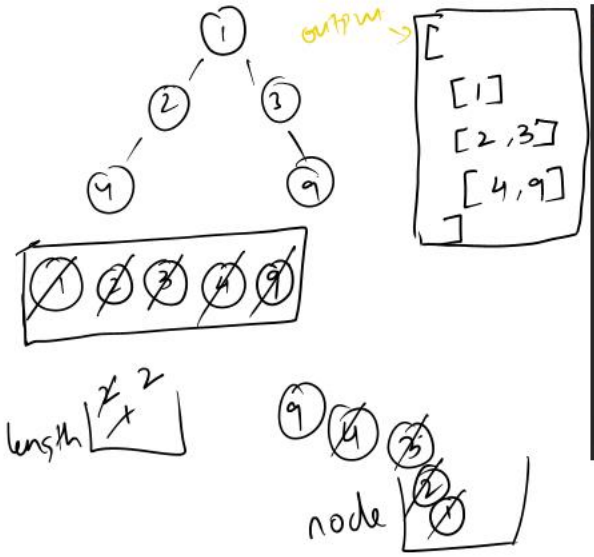
Medium | Topics | Companies | Hint

Given the `root` of a binary tree, return *the level order traversal of its nodes' values*. (i.e., from left to right, level by level).









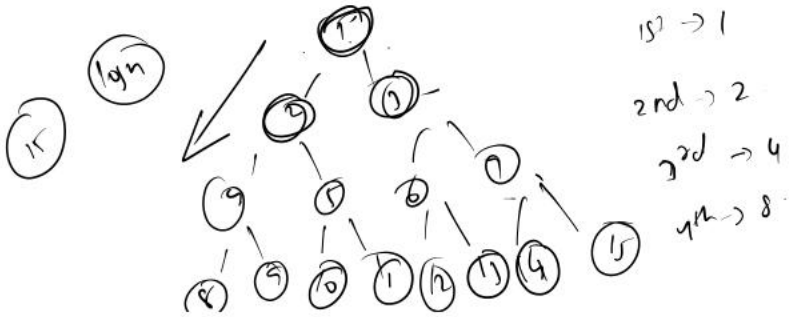

```python
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        output = []
        queue = deque()
        queue.append(root)
        while queue :
            length = len(queue)
            current = []
            for i in range(length) :
                node = queue.popleft()
                current.append(node.val)
                if node.left :
                    queue.append(node.left)
                if node.right :
                    queue.append(node.right)
            output.append(current)
        return output
```

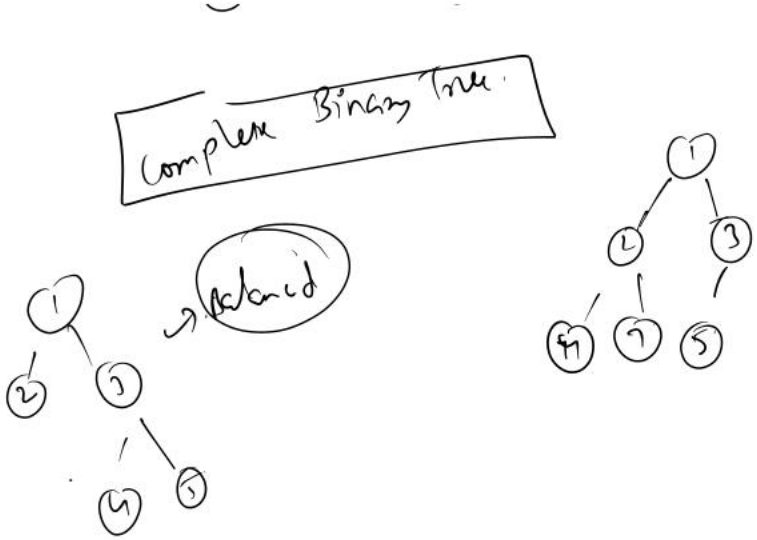
```python
class Solution:
    def levelOrder(self, root: Optional[TreeNode]) -> List[List[int]]:
        if root is None :
            return []
        output = []
        queue = deque()
        queue.append(root)
        while queue :
            length = len(queue)
            current = []
            for i in range(length) :
                node = queue.popleft()
                current.append(node.val)
                if node.left :
                    queue.append(node.left)
                if node.right :
                    queue.append(node.right)
            output.append(current)
        return output
```

T.C -> O(n)

n -> no of nodes

S.C -> O(2^n log n)





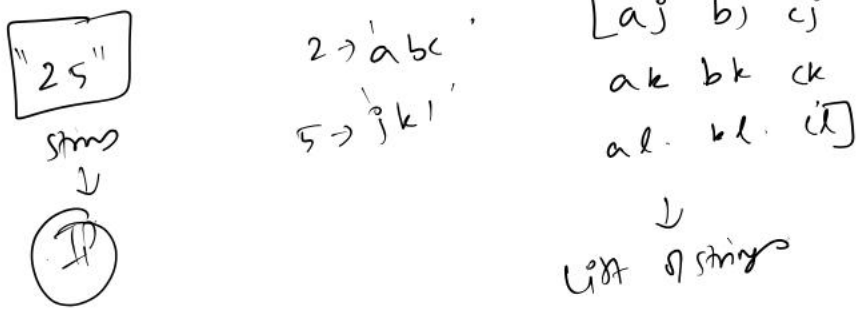
17. Letter Combinations of a Phone Number

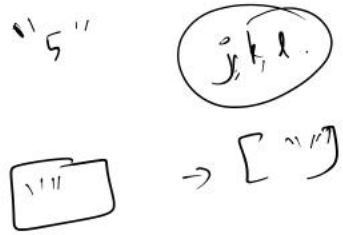
Solved

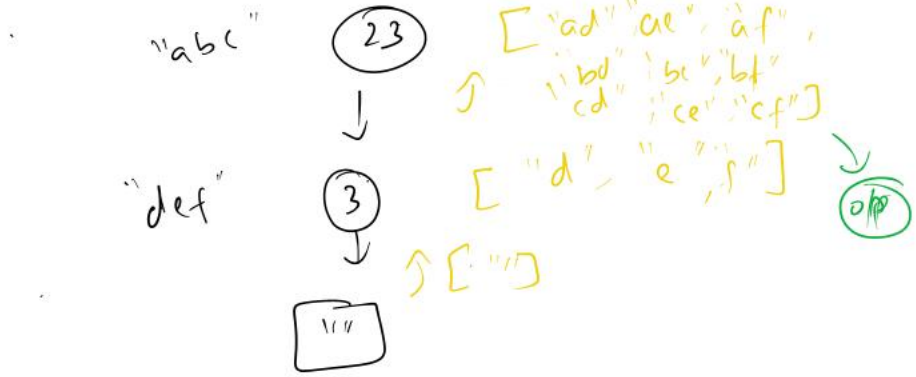
Medium | Topics | Companies

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

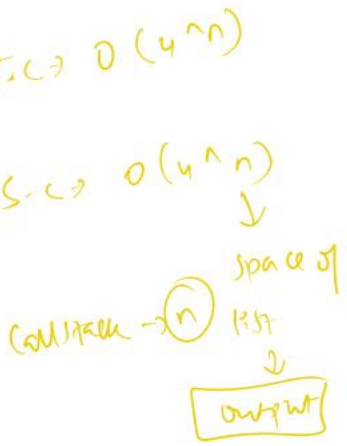
A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.








```python
class Solution:
    def letterCombinations(self, digits: str) -> List[str]:
        def helper(digits) :
            if len(digits) == 0 :
                return ['']
            smallOutput = helper(digits[1 : ])
            output = []
            options = map[digits[0]]
            for i in options :
                for s in smallOutput :
                    output.append(i + s)
            return output
        map = {
            '2' : 'abc',
            '3' : 'def',
            '4' : 'ghi',
            '5' : 'jkl',
            '6' : 'mno',
            '7' : 'pqrs',
            '8' : 'tuv',
            '9' : 'wxyz'
        }
        return helper(digits)
```



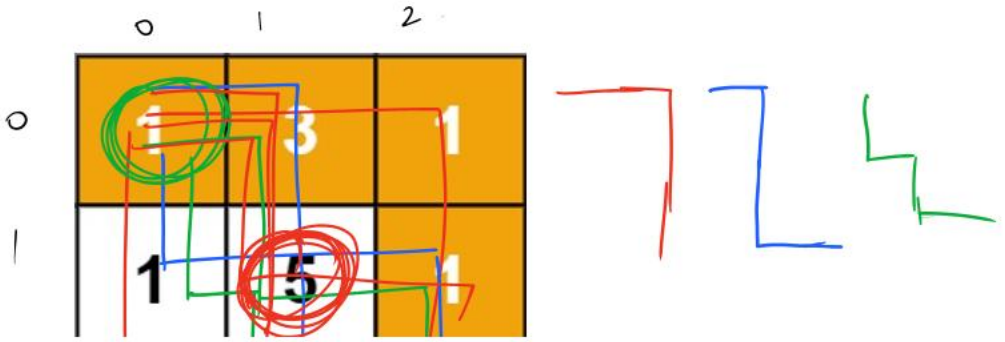
64. Minimum Path Sum

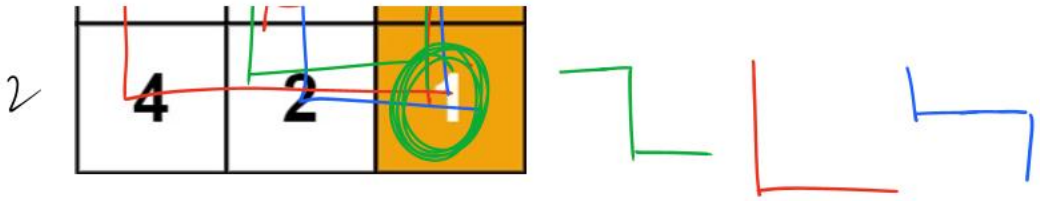
Solved

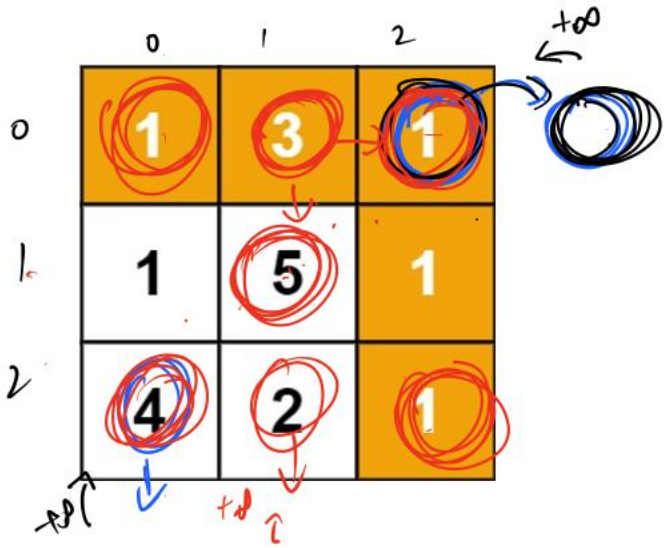
Medium | Topics | Companies

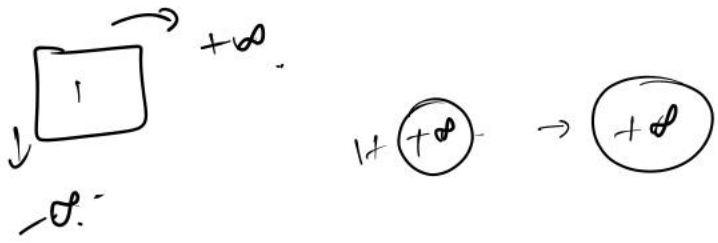
Given a `m x n` `grid` filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

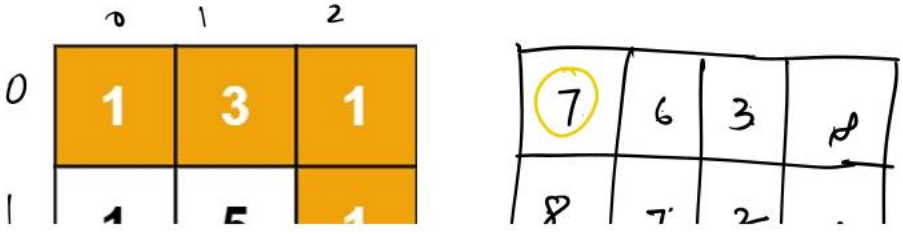


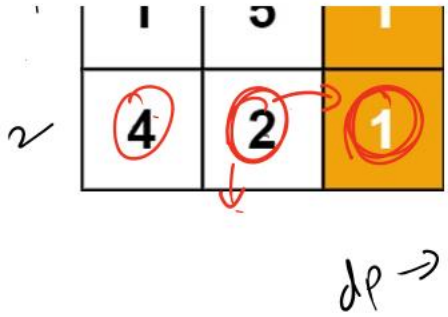


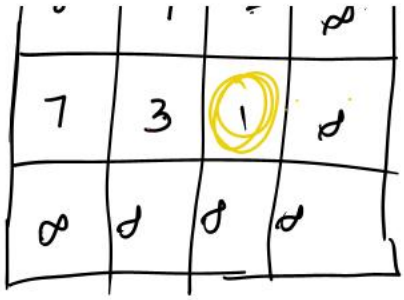





```python
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        def helper(i, j) :
            if i == m or j == n :
                return float('inf')
            if i == m - 1 and j == n - 1 :
                return grid[i][j]
            rightAns = helper(i, j + 1)
            downAns = helper(i + 1, j)
            return grid[i][j] + min(rightAns, downAns)
        m = len(grid)
        n = len(grid[0])
        return helper(0, 0)
```








```python
class Solution:
    def minPathSum(self, grid: List[List[int]]) -> int:
        m = len(grid)
        n = len(grid[0])
        dp = [[float('inf')] * (n + 1) for x in range(m + 1)]
        for i in range(m - 1, -1, -1) :
            for j in range(n - 1, -1, -1) :
                if i == m - 1 and j == n - 1 :
                    dp[i][j] = grid[i][j]
                    continue
                dp[i][j] = grid[i][j] + min(dp[i][j + 1], dp[i + 1][j])
        return dp[0][0]
```

S.C -> O(m×n)

T.C -> O(m×n).

knapsack 01 practice

Byte Landian

Rat in a Maze

Sorting

Two pointer

Prefix Sum

sliding window

Floyd's cycle

LL

Trees

BackTracking

DP

Graphs

Stack Queues

Dictionary

+

set

P.Name Type Link Description Approach T.C - S.C